4 p^{tos}

1. Dada uma lista de N inteiros não repetidos e k centros, inteiros dessa lista, pretende-se criar k sublistas (k >= 2), de tal modo que em cada sublista ficam os inteiros mais próximos do respectivo centro, ou seja, os inteiros da lista inicial com a menor diferença absoluta relativamente ao centro.
Como exemplo considere a seguinte lista L = { 2, 9, 7, 5, 10, 15, 6, 12, 3} e os centros C_1=3, C_2=6 e C_3=10. A lista original L será dividida nas três sublistas: L1 = {3, 2}, L2 = {6, 7, 5} e L3 = {10, 9, 15, 12} que devem ser devolvidas num contentor map que guarda os pares <centro, sublista>.

```
Map<Integer, LinkedList<Integer>> KsubLists(LinkedList<Integer> list,
                                                    ArrayList<Integer> centers)
```

3 p^{tos}

2. Considere os seguintes métodos:

```
public static void mistery1 (Stack<Integer> S){

    if (!S.isEmpty()){
        Integer elem=S.pop();
        mistery1(S);
        S.push(elem);
    }
}

public static void mistery2 (Queue<Integer> q){

    if (!q.isEmpty()){
        Integer elem =q.remove();
        mistery2(q);
        q.add(elem);
    }
}
```

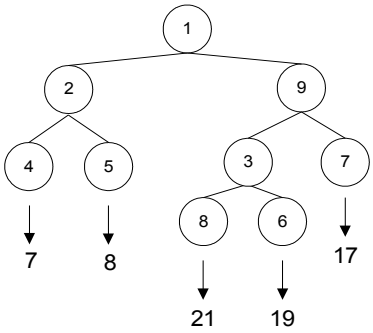
a) Explique o que fazem os métodos acima apresentados.

b) Analise-os quanto à sua complexidade temporal. Justifique.

4.5 p^{tos}

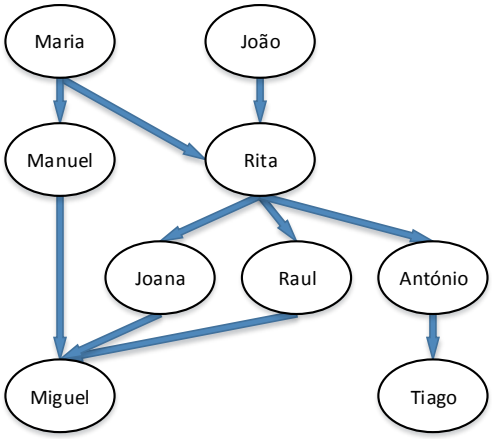
3. Adicione à classe TREE<E> um método que devolve numa lista a soma dos conteúdos dos nós de todos os caminhos desde o nó raiz até às folhas.

   Exemplo:



4.5 p^{tos}

4. Considere um grafo orientado e acíclico, onde cada vértice representa um funcionário e cada ramo x -> y representa semanticamente que o funcionário y somente pode ser promovido se o funcionário x também for. Por exemplo, no grafo da figura se pretendermos encontrar 2 promoções, há 2 listas possíveis: { Maria, João } ou { Maria, Manuel }. No caso de se pretenderem 3 promoções, as opções são { Maria, João, Manuel } ou { Maria, João, Rita }.



Usando a representação **map de adjacência** implemente um método que, dado um grafo e um número **n** de promoções, devolva uma lista com os funcionários a promover. O objetivo é somente encontrar uma das soluções. O método a desenvolver deve obedecer à interface:

```
List<String> calculaPromocoes(Graph<String,Integer> g, Integer n)
```

4 p^{tos}

5. Usando os métodos da classe HeapPriorityQueue<K,V> escreva um método eficiente que faz a junção ordenada de duas heaps num vector.